Advanced Terraform on AWS Lab 3 Dynamic Resource Creation and Deterministic Design - Challenge

Item	Details
Expected Duration	60-70 minutes
Lab Format	Challenge
Starting Point	Files based on Lab 2 Phase 4 enforcement with additional resources added.
Exemplar Solution	Available after you have completed or attempted the challenge

Challenge Sections

- Scenario
- What You Are Trying to Do
- Existing Configuration Review
- Challenge Requirements
- Validation Tests
- Deliverables
- Review Questions
- Lab Clean-up

Scenario

Most of the code provided in this lab follows the engineered Terraform pattern established in Lab 2. The security group configuration already demonstrates structured input, data preparation, validation, sanitization, dynamic resource creation and deterministic resource identity.

A new networking section has been introduced creating subnets and route table associations. It works, but it has been written in a more manual style. Your task is to refactor that networking section, so it follows the same Terraform design principles used elsewhere in the codebase.

What You Are Trying to Do

This is **not** an AWS networking exercise. The new subnet and route table resources are the vehicle for practising Terraform design. Your goal is to take a working but repetitive implementation and turn it into a scalable, deterministic, pattern-driven Terraform configuration.

- Recognise where the current networking implementation does not follow the established codebase pattern.
- Replace isolated subnet inputs with a structured input model.
- Remove repetitive subnet resource definitions.
- Remove repetitive route table association definitions.
- Ensure dependent resources scale automatically from the subnet model.
- Demonstrate that resource identity remains stable when subnet definitions are reordered.
- Ensure cosmetic formatting differences in subnet input do not create infrastructure churn.
- Keep resource creation logic separate from data preparation logic.

Existing Configuration Review

Before making any changes, review the starting configuration.

- Open main.tf and locate the subnet resources and route table association resources.
- Open variables.tf and terraform.tfvars and locate the current subnet input variables.
- Compare the networking section with the existing security group section.
- Identify which parts of the networking implementation are repetitive or difficult to scale.
- Identify which Terraform patterns are already being used by the security group implementation.

Baseline Deployment

Deploy the starting configuration to confirm that the original implementation works before refactoring it.

```
terraform init
terraform plan
terraform apply --auto-approve
terraform state list
```

You should see the existing Lab 2 resources along with the new networking resources introduced for this lab.

Controlled Rollback

Before refactoring, remove only the networking resources introduced in this lab. Leave the rest of the infrastructure in place.

```
terraform destroy `
  -target=aws_subnet.app `
  -target=aws_subnet.data `
  -target=aws_route_table_association.app `
  -target=aws_route_table_association.data `
  -target=aws_route_table.main `
  -auto-approve
```

Challenge Requirements

Refactor the networking configuration so that it satisfies the following requirements. The requirements describe the desired behaviour, not the exact implementation steps.

Requirement	Description	Success Indicator
1	Subnet definitions must be managed through a single structured input value rather than separate variables for each subnet.	Adding a subnet should require an input-data change, not a new variable.
2	The subnet CIDR input must reject malformed CIDR blocks while tolerating harmless whitespace.	Invalid CIDR values fail validation; values with leading or trailing spaces can still be handled safely.
3	Subnet resources must be generated from the structured subnet data rather than manually duplicated.	Adding a new subnet does not require an additional subnet resource block.
4	Route table associations must scale automatically with the subnets they depend on.	Adding a subnet automatically creates a matching route table association.
5	Subnet identity must be stable and meaningful.	Reordering subnet definitions does not cause existing subnets to be replaced.

6	Cosmetic formatting differences must not leak into the infrastructure model.	Whitespace changes in subnet names or CIDR values should not produce unnecessary changes.
7	Existing security group behaviour must remain unchanged.	The existing security group rules still plan and apply as before.

Constraints

- Do not redesign the VPC or security group implementation unless a change is directly required by this challenge.
- Keep the same VPC CIDR block and region behaviour as the starting configuration.
- Keep the route table as a single shared route table for the lab.
- Use the same naming pattern already established by the codebase.
- The final configuration should still be readable by someone reviewing the code after the lab.

Validation Tests

Use the following tests to prove that your refactor behaves correctly. Your final solution should pass all tests.

Test 1 - Recreate the Two Original Subnets

After your initial refactor, run:

```
terraform plan
```

Expected result: Terraform should propose the same intended networking model as the original implementation but using your refactored design.

Test 2 - Apply the Refactored Design

```
terraform apply --auto-approve
terraform state list
```

Expected result: the state should show subnet and route table association instances that are tied to stable subnet identities rather than to separate hard-coded resource names.

Test 3 - Add a Management Subnet

Extend the subnet input data so that it includes a third subnet named mgmt using the CIDR block 10.50.3.0/24.

```
terraform plan
terraform apply --auto-approve
```

Expected result: Terraform should add one new subnet and one new route table association. Existing subnet resources should remain unchanged.

Test 4 - Reorder the Subnet Definitions

Reorder the subnet definitions in the input file so that the entries appear in a different order.

```
terraform plan
```

Expected result: Terraform should propose no infrastructure changes. This proves that resource identity is not based on positional ordering.

Test 5 - Introduce Harmless Whitespace

Introduce leading and trailing spaces around some subnet names or CIDR values.

```
terraform plan
```

Expected result: Terraform should propose no infrastructure changes. This proves that formatting inconsistencies are handled before resource creation.

Deliverables

When you have completed the challenge, be prepared to show or explain the following:

- The final subnet input structure.
- The final subnet resource design.
- The final route table association design.
- Where CIDR validation occurs.
- Where subnet input sanitization occurs.
- Why reordering subnet definitions does not cause infrastructure churn.
- Why adding a subnet requires only an input-data change.
- A successful final terraform plan showing no unexpected changes.

Review Questions

- What was the main weakness of the original subnet implementation?
- Why is a structured input model easier to maintain than separate variables?
- How does Terraform decide the identity of dynamically generated resources in your solution?
- Why is stable resource identity important in infrastructure-as-code?

- How does the route table association design scale with the subnet design?
- Why is it useful to sanitize input before resource creation?
- How does this refactor reflect the patterns already used by the security group implementation?

Exemplar Solution

An exemplar solution is available separately. Do not refer to it until you have either completed the challenge or made a serious attempt at the refactor. The purpose of this challenge is not simply to produce working AWS resources, but to practise recognising and applying Terraform design patterns.

Lab Clean-up

When the challenge is complete, destroy the lab resources:

```
terraform destroy --auto-approve
```

Congratulations, you have completed this challenge lab

Copyright